CICM — Component Introduction & Change Module

CICM — Component Introduction & Change Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Dependency Standard (SDS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Component Introduction & Change Governance

Module ID: SLGA-SDS-CICM-004

Origin ID: OOF.CM.SLGA.03.04.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Component Introduction & Change Module (CICM) governs the legitimate

introduction, inclusion, modification, update, replacement, restriction, and removal of software dependencies within a defined software system and its governed lifecycle context.

CICM establishes the methodological conditions under which a software

component or dependency relationship may be introduced or changed, based on applicable authority, defined scope, established dependency identity, origin and traceability, suitability, relevant constraints, evidence, and the material consequences of the proposed action.

It governs the relationship between a dependency action, the actor or

mechanism performing that action, the authority under which it is permitted, the specific component and dependency relationship affected, the applicable conditions, the resulting software state, and the evidence necessary to reconstruct what was authorized and what actually occurred.

CICM distinguishes technical capability from legitimate authority, approval

of a development objective from permission for each material dependency action, proposed dependency change from authorized dependency change, and successful technical execution from governed acceptance of the resulting dependency state.

It applies to dependency actions performed by human developers,

organizations, automated development tools, AI coding assistants, autonomous software development agents, build systems, package managers, integration processes, and other software execution mechanisms.

Its principal output is:

Governed Component Introduction & Change State

Its foundational distinction is:

Dependency Modification Capability ≠ Dependency Modification Authority.

Operational Role

The operational role of CICM is:

To ensure that material software dependency introductions and changes occur only within established authority, scope, conditions, and evidence-supported governance boundaries, while preserving the relationship between the authorized action, its actual execution, and the resulting dependency state.

CICM establishes the action-governance foundation of SDS.

It determines whether a proposed or executed dependency action is legitimate within the applicable governance context.

It does not independently determine component identity, establish complete provenance, certify suitability, guarantee security, or govern the entire software change-management lifecycle.

It relies on competent upstream assessments and preserves the

dependency-specific action conditions necessary for downstream governance.

Operational Question

Under whose authority, within which scope and conditions, and on the basis of what evidence may a software dependency be introduced, modified, updated, replaced, restricted, or removed—and how can the actual action and resulting dependency state remain traceable to that authority?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

CICM operates within:

Software Dependency Standard (SDS)

The SDS internal governance flow is:

Component Identification → Dependency Origin & Traceability → Dependency

Suitability → Component Introduction & Change → Dependency Continuity & Risk

CICM governs:

Component Introduction & Change

It is the fourth Core Module of SDS.

Its principal upstream inputs are:

Governed Software Component Identification State

from:

CIM — Component Identification Module

Governed Dependency Origin & Traceability State

from:

DOTM — Dependency Origin & Traceability Module

Governed Dependency Suitability State

from:

DSM — Dependency Suitability Module

Its principal output is:

Governed Component Introduction & Change State

This output supports:

DCRM — Dependency Continuity & Risk Module

and competent downstream SLGA™ governance.

CICM may be invoked whenever a material dependency action is proposed, executed, discovered, corrected, reversed, or reassessed.

• Governed Space

Software Component Introduction & Change Governance

This space governs the legitimacy, scope, execution relationship, evidence,

and resulting state of material software dependency actions.

Its principal dimensions include:

• Dependency action governance objects

• Dependency action identification

• Actor and execution mechanism identification

• Authority source

• Authority validity

• Authority scope

• Authority conditions

• Dependency action permission

• Component introduction

• Dependency inclusion

• Dependency modification

• Dependency update

• Dependency replacement

• Dependency restriction

• Dependency removal

• Direct and transitive change effects

• Autonomous dependency actions

• Constituent action authority

• Dependency change proposals

• Dependency action authorization

• Execution traceability

• Actual-versus-authorized action comparison

• Resulting dependency state

• Action evidence

• Authorization evidence

• Action uncertainty

• Authority conflicts

• Unauthorized action recognition

• Action correction and reversal

• Change continuity

• Dependency action governance states

The governed space is independent of programming language, package manager, repository, software development methodology, or actor type.

Component Introduction & Change Governance Object

Every CICM assessment shall establish a Component Introduction & Change

Governance Object.

The object represents a proposed, authorized, executed, or discovered

dependency action.

It should identify:

Governance Object Identity: The identifiable dependency action assessment.

Dependent Software Object: The software system affected.

Dependency Component: The component or resource affected.

Dependency Relationship: The reliance relationship being introduced or

changed.

Action Type: The intended or observed dependency action.

Actor or Mechanism: The human, organization, software tool, or autonomous

system performing the action.

Authority Reference: The claimed or established basis for the action.

Authority Scope: The permitted boundaries of the action.

Action Conditions: Applicable prerequisites and restrictions.

Upstream Governance References: Relevant component identification,

provenance, and suitability states.

Execution Evidence: Evidence of the action actually performed.

Resulting Dependency State: The resulting component and relationship

condition.

Action Uncertainty: Material unresolved action or authority conditions.

Governance State: The recognized dependency action condition.

The governance object shall remain distinguishable from the dependency

artifact and the software change record.

Dependency Action Governance Object ≠ Dependency Artifact.

Dependency Action Identification

CICM shall identify the material dependency action under assessment.

Relevant actions may include:

• Introducing a dependency

• Including a component

• Installing a package

• Updating a version

• Replacing a component

• Modifying a dependency

• Restricting dependency use

• Removing a dependency

• Reconfiguring a dependency relationship

• Changing a dependency source

• Changing a dependency resolution mechanism

An action shall be identified sufficiently to distinguish it from other

dependency operations.

Dependency Action Name ≠ Complete Dependency Action Identity.

Dependency Action Purpose

CICM shall establish the purpose of the proposed dependency action.

The purpose should identify:

• The software requirement or operational need

• The intended dependency effect

• The affected software object

• The relevant component

• The expected resulting condition

The existence of a useful action shall not independently establish

authority to perform it.

Useful Dependency Action ≠ Authorized Dependency Action.

Actor Identification

CICM shall identify the actor or mechanism responsible for performing the

dependency action.

Relevant actors may include:

• Human developers

• Software maintainers

• Development teams

• Organizations

• Automated build systems

• Package managers

• AI coding assistants

• Autonomous development agents

• Integration services

• Supplier-controlled mechanisms

Actor identity shall remain distinguishable from authority.

Actor Identification ≠ Action Authorization.

Execution Mechanism Identification

CICM shall identify the mechanism through which a dependency action is performed.

Mechanisms may include:

• Manual installation

• Package manager commands

• Automated dependency resolution

• Build scripts

• Continuous integration pipelines

• AI tool calls

• Autonomous agent workflows

• Runtime component loading

• Supplier updates

• Deployment automation

Technical execution mechanisms shall not independently establish legitimate

authority.

Execution Mechanism Capability ≠ Governance Permission.

Dependency Action Authority

CICM shall establish the applicable authority basis for a material

dependency action.

Authority may arise through:

• Organizational delegation

• Defined software governance responsibilities

• Approved development scope

• Authorized maintenance responsibilities

• Established operational permissions

• Competent lifecycle governance decisions

Authority shall be evaluated relative to the particular action, software

object, scope, and conditions.

General Development Authority ≠ Unlimited Dependency Action Authority.

Authority Source Identification

CICM shall identify the source from which the claimed dependency action

authority originates.

Relevant sources may include:

• Organizational authority structures

• Delegated responsibilities

• Approved development instructions

• Authorized change decisions

• Established operational policies

• Competent software governance determinations

A recorded instruction shall not automatically be considered a valid

authority source.

Recorded Instruction ≠ Legitimate Authority.

Authority Validity

CICM shall assess whether the claimed authority remains applicable to the

proposed dependency action.

Relevant considerations may include:

• Authority origin

• Current validity

• Delegation conditions

• Action type

• Software scope

• Temporal limits

• Applicable restrictions

• Revocation or supersession

A previously valid authority may no longer apply.

Historical Authorization ≠ Current Authorization.

Authority Scope

CICM shall establish the boundaries within which a dependency action may be performed.

Scope may concern:

• Software project

• Software subsystem

• Dependency category

• Component identity

• Component version

• Development environment

• Production environment

• Action type

• Permitted technical operations

An actor may be authorized to modify one dependency without being authorized to introduce unrelated dependencies.

Authority for One Dependency ≠ Authority for Every Dependency.

Authority Conditions

CICM shall preserve conditions attached to dependency action authority.

Conditions may include:

• Required upstream assessments

• Defined component versions

• Restricted repositories

• Approved dependency categories

• Required testing

• Review requirements

• Evidence preservation

• Environmental restrictions

• Additional approval thresholds

Authority shall not be treated as unconditional where material conditions apply.

Conditional Authority ≠ Unrestricted Authority.

Constituent Action Authority

CICM shall distinguish authority for an overall development objective from

authority for the individual dependency actions used to achieve it.

A software development task may involve:

• Searching for components

• Downloading packages

• Accessing repositories

• Executing installation commands

• Modifying build files

• Changing dependency versions

• Introducing external services

• Removing existing components

Each material constituent action may require a distinct authority determination.

Task Authority ≠ Constituent Action Authority.

Component Introduction

CICM shall govern the introduction of a previously absent dependency into

the relevant software context.

Introduction assessment should consider:

• Component identity

• Origin and traceability

• Suitability

• Introduction purpose

• Applicable authority

• Action scope

• Material constraints

• Expected resulting dependency state

A dependency shall not be treated as legitimately introduced solely because

it has been installed.

Successful Component Installation ≠ Governed Component Introduction.

Dependency Inclusion

CICM shall govern the inclusion of software components within a software artifact or dependent software environment.

Inclusion may occur through:

• Package installation

• Source embedding

• Static linking

• Dynamic linking

• Service integration

• Generated code incorporation

• Build artifact composition

Inclusion shall remain distinguishable from component identification and

from authority to include it.

Component Inclusion ≠ Authorized Component Inclusion.

Dependency Modification

CICM shall govern material modification of an existing dependency.

Modification may include:

• Source changes

• Local patches

• Configuration-related dependency changes

• Interface modifications

• Component rebuilding

• Customized distributions

• Altered dependency relationships

Modification authority shall be established for the relevant component and

action scope.

Ability to Modify a Dependency ≠ Authority to Modify It.

Dependency Update

CICM shall govern material dependency updates.

An update may change:

• Component version

• Functional behavior

• Interfaces

• Runtime requirements

• Transitive dependencies

• Distribution origin

• Resource requirements

• Compatibility conditions

An update shall not automatically be considered authorized merely because

it is available or recommended by a tool.

Available Dependency Update ≠ Authorized Dependency Update.

Dependency Replacement

CICM shall govern replacement of an existing dependency with another

component or service.

Replacement assessment should identify:

• Existing dependency

• Proposed replacement

• Replacement purpose

• Relevant suitability conditions

• Applicable authority

• Affected dependency relationships

• Material transition conditions

• Expected resulting state

Functional similarity shall not independently establish legitimate

replacement.

Functional Similarity ≠ Governed Dependency Equivalence.

Dependency Restriction

CICM shall govern actions that restrict the use or scope of a dependency.

Restrictions may concern:

• Permitted software environments

• Component versions

• Runtime functions

• Interface access

• Distribution channels

• Dependency categories

• Operational conditions

Restriction shall remain traceable to applicable authority and scope.

Technical Restriction Capability ≠ Legitimate Restriction Authority.

Dependency Removal

CICM shall govern material dependency removal.

Removal assessment should consider:

• The dependency relationship being removed

• The affected software object

• Relevant direct and transitive relationships

• Remaining embedded or runtime components

• Replacement conditions

• Applicable authority

• Resulting dependency state

Removal from a manifest shall not automatically establish complete

dependency elimination.

Manifest Removal ≠ Dependency Elimination.

Direct Dependency Change Effects

CICM shall identify material direct effects of dependency actions.

Effects may include:

• Changed component versions

• New interfaces

• Removed functionality

• Modified build requirements

• Changed runtime conditions

• Altered configuration references

Direct effects shall be sufficiently identified for competent downstream

governance.

Successful Direct Change ≠ Complete Change Impact Recognition.

Transitive Dependency Change Effects

CICM shall recognize material changes introduced indirectly through dependency actions.

An update to one package may alter multiple transitive dependencies.

CICM shall preserve the relationship between the initiating action and

materially affected indirect components.

Direct Update Authorization ≠ Automatic Authorization of Every Transitive

Change.

Where applicable authority explicitly covers bounded transitive changes, those conditions shall be preserved.

Dependency Change Scope

CICM shall establish the scope of a proposed dependency change.

Scope should identify:

• Affected software objects

• Target components

• Permitted versions

• Relevant environments

• Authorized operations

• Material dependency relationships

• Expected resulting conditions

An action exceeding the established scope shall not be treated as

authorized merely because it serves the same development objective.

Shared Development Objective ≠ Shared Change Authority.

Dependency Change Proposal

CICM shall distinguish a proposed dependency action from an authorized

action.

A proposal should identify:

• Proposed action

• Purpose

• Affected components

• Relevant upstream governance references

• Expected effects

• Material constraints

• Required authority

• Supporting evidence

A proposal shall remain a proposal until the applicable authorization

conditions are satisfied.

Proposed Dependency Change ≠ Authorized Dependency Change.

Dependency Action Authorization

CICM shall determine whether the proposed dependency action falls within established authority, scope, and conditions.

Authorization assessment should consider:

• Actor or mechanism

• Authority source

• Authority validity

• Action type

• Affected dependency

• Software scope

• Upstream governance states

• Material restrictions

• Required evidence

• Applicable decision conditions

Authorization shall remain limited to the action and context assessed.

Authorized Dependency Action ≠ Unlimited Future Dependency Authority.

Authorization Evidence

CICM shall preserve evidence supporting dependency action authorization.

Evidence may include:

• Approved change records

• Delegation references

• Competent governance decisions

• Authorized development instructions

• Defined operational permissions

• Applicable policy records

• Approval conditions

The existence of an approval record shall not independently establish that the actual action complied with it.

Authorization Record ≠ Authorized Execution.

Pre-Execution Governance Conditions

CICM shall identify material conditions that must be satisfied before a dependency action is executed.

Conditions may include:

• Sufficient component identification

• Relevant provenance assessment

• Suitability determination

• Authority validation

• Change scope definition

• Required testing

• Evidence requirements

• Competent escalation

Unresolved material conditions shall remain explicit.

Technical Readiness ≠ Governance Readiness.

Dependency Action Execution

CICM shall preserve the relationship between an authorized dependency

action and its actual execution.

Execution evidence should identify:

• Action performed

• Actor or mechanism

• Affected component

• Relevant software object

• Execution context

• Material parameters

• Resulting technical changes

• Relevant execution evidence

An executed action shall not be presumed authorized.

Executed Dependency Action ≠ Authorized Dependency Action.

Authorized-versus-Actual Action Comparison

CICM shall assess whether the actual dependency action corresponds to the

authorized action.

Relevant comparisons may include:

• Authorized component versus actual component

• Authorized version versus installed version

• Authorized repository versus retrieval source

• Authorized environment versus affected environment

• Authorized action type versus executed action

• Authorized scope versus actual effects

Material deviations shall remain explicit.

Authorized Intent ≠ Actual Execution Conformity.

Resulting Dependency State

CICM shall identify the resulting dependency condition after a material

action.

The resulting state may include:

• Newly introduced dependency

• Updated dependency

• Modified dependency

• Replaced dependency

• Restricted dependency

• Removed dependency

• Partially executed change

• Failed change

• Unresolved change effects

The resulting technical state shall remain distinguishable from its

governance acceptance.

Resulting Technical State ≠ Governed Dependency Change State.

Dependency Change Evidence

CICM shall preserve evidence supporting material dependency action claims.

Evidence may include:

• Change requests

• Authorization records

• Package manager logs

• Repository commits

• Build records

• Artifact differences

• Agent tool-call records

• Runtime observations

• Dependency inventory changes

• Relevant test results

Evidence shall remain associated with the specific action or claim it

supports.

Change Record Presence ≠ Complete Dependency Action Traceability.

Change Evidence Competence

CICM shall assess whether available evidence is competent to support the claimed action, authority, or result.

For example:

An approval record may support an authorization decision.

A package manager log may support an installation event.

A repository commit may support a recorded source change.

A runtime observation may support actual component use.

These records establish different facts.

Evidence of Execution ≠ Evidence of Authorization.

Change Evidence Integrity

CICM shall preserve the integrity of material dependency action evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Temporal relevance

• Actor correspondence

• Component correspondence

• Action correspondence

• Preservation history

• Material contradictions

Evidence integrity does not independently establish action legitimacy.

Intact Execution Evidence ≠ Legitimate Execution.

Autonomous Dependency Action Governance

CICM shall govern material dependency actions performed by autonomous

software development systems.

An autonomous agent may:

• Discover packages

• Download components

• Install libraries

• Modify dependency manifests

• Execute build commands

• Update packages

• Replace components

• Remove dependencies

• Introduce service integrations

The agent's technical capability shall not independently establish

authority for these actions.

Autonomous Execution Capability ≠ Autonomous Execution Authority.

Autonomous Tool-Path Authority

CICM shall preserve the distinction between authority for an autonomous

development objective and authority for the tool path used to achieve it.

A relevant path may be:

Development Objective → Authority → Tool Selection → Package Retrieval →

Installation → Software Modification → Resulting Dependency State

Each material constituent action shall remain subject to applicable

authority conditions.

An authorized objective shall not automatically legitimize every tool call,

retrieval source, or software modification.

Task Authority ≠ Constituent Action Authority.

Authorized Objective ≠ Authorized Tool Path.

Automated Dependency Updates

CICM shall govern material dependency updates performed through automated mechanisms.

Relevant mechanisms may include:

• Automated package updates

• Dependency update bots

• Continuous integration workflows

• Scheduled maintenance processes

• Autonomous software maintenance agents

Automation shall operate within defined authority, scope, and conditions.

An automated update process shall not independently expand its own

authority.

Automated Update Configuration ≠ Unlimited Update Authorization.

Dependency Action Delegation

CICM shall preserve material delegation relationships affecting dependency

actions.

Delegation may concern:

• Human maintainers

• Development teams

• Software services

• Automated tools

• AI agents

• Autonomous maintenance systems

Delegated authority shall remain bounded by its source, scope, and

conditions.

A delegated actor shall not be presumed entitled to further delegate

authority.

Delegated Execution Responsibility ≠ Unlimited Delegation Authority.

Unauthorized Dependency Action

An Unauthorized Dependency Action exists where a material dependency action occurs without sufficiently established applicable authority.

Examples may include:

• Installing an unapproved external library

• Updating a dependency outside permitted scope

• Replacing a component without required authorization

• Introducing an external service through an autonomous tool call

• Removing a critical dependency without competent approval

CICM shall preserve the action, evidence, and authority gap.

Unauthorized action recognition does not independently establish malicious intent or legal liability.

Unauthorized Dependency Action ≠ Proven Malicious Action.

Authority Uncertainty

Authority uncertainty may arise where:

• The authority source is unclear

• Delegation conditions are incomplete

• Scope is ambiguous

• Approval validity is uncertain

• An autonomous agent exceeds an unclear permission boundary

• A material transitive change is not explicitly covered

CICM shall preserve material uncertainty.

Unresolved Authority ≠ Established Authorization.

Authority Conflict

An authority conflict exists where material authority claims or conditions are incompatible.

Examples may include:

• Conflicting approval instructions

• Incompatible delegation scopes

• Contradictory organizational policies

• Disagreement between development and operational authority

• Conflicting permissions for autonomous agents

CICM shall preserve the conflict and support competent escalation.

Conflicting Authority Claims ≠ Permission to Select the Most Convenient

Authority.

Dependency Action Gap

A Dependency Action Gap exists where material action-governance information is missing.

Examples may include:

• Missing authority evidence

• Unknown actor

• Incomplete execution record

• Unclear affected component

• Unresolved action scope

• Missing resulting-state evidence

• Untraceable autonomous tool actions

CICM shall identify the gap and its material significance.

Action Evidence Gap ≠ Proven Unauthorized Action.

Critical Dependency Action Gap

A Critical Dependency Action Gap exists where missing action or authority evidence prevents competent governance of a materially consequential dependency change.

Examples may include:

• An unidentified actor replacing a critical dependency

• An autonomous agent introducing an external component without

• A production dependency update with unresolved scope

• A critical component removal without sufficient resulting-state

reconstructable authorization. evidence.

CICM shall preserve the critical gap for competent assessment.

It does not independently determine software suspension, security compromise, or legal liability.

Critical Action Gap ≠ Automatic Security Incident.

Dependency Action Correction

CICM shall support governance of corrections to material dependency actions.

Corrections may include:

• Reversing an unauthorized update

• Restoring an earlier dependency version

• Replacing an incorrectly introduced component

• Correcting an incomplete dependency removal

• Restricting an improperly introduced service integration

Corrective actions shall themselves remain subject to applicable authority.

Corrective Intent ≠ Corrective Action Authority.

Dependency Action Reversal

CICM shall govern reversal of material dependency actions where applicable.

Reversal assessment should consider:

• Original action

• Resulting dependency state

• Reversal authority

• Technical feasibility

• Affected dependency relationships

• Material side effects

• Evidence of actual reversal

A reversal command shall not automatically establish restoration of the

prior dependency state.

Rollback Execution ≠ Verified State Restoration.

Dependency Change Continuity

CICM shall preserve material relationships between successive dependency

actions.

Continuity may involve:

• Initial introduction

• Version updates

• Local modifications

• Supplier changes

• Component replacement

• Restrictions

• Removal

• Reintroduction

Historical dependency actions shall remain reconstructable where material.

Current Dependency State ≠ Complete Dependency Change History.

Dependency Action Governance States

CICM shall support identifiable governance states.

Possible states include:

Proposed: A dependency action has been requested or identified.

Under Assessment: Authority and action conditions are being evaluated.

Authorized: The action is permitted within defined scope and conditions.

Conditionally Authorized: Permission depends on explicit conditions.

Authority-Unresolved: Applicable authority is insufficiently established.

Scope-Unresolved: Material action boundaries remain unclear.

Evidence-Limited: Available evidence is insufficient for the required determination.

Execution-Pending: An authorized action has not yet been executed.

Executed: The action has technically occurred, without implying

governance conformity.

Execution-Conformant: The actual action sufficiently corresponds to its

authorization.

Execution-Deviated: Material execution conditions differ from the

authorized action.

Unauthorized: The action has been determined to lack applicable authority.

Conflicted: Material authority or action evidence is contradictory.

Correction-Required: Competent governance has established a need for

corrective action.

Reversed: A material action has been reversed within the assessed scope.

Superseded: A later governance state has replaced the earlier state.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not independently establish software quality, security,

or operational continuity.

Dependency Action Authorized ≠ Software Product Accepted.

Governed Component Introduction & Change State

The principal output of CICM is:

Governed Component Introduction & Change State

The state shall preserve sufficient information to identify:

Action Governance Object: The dependency action assessment.

Dependent Software Object: The software system affected.

Dependency Component: The component involved.

Dependency Relationship: The material reliance relationship.

Action Type: The introduction or change operation.

Action Purpose: The intended reason for the action.

Actor or Mechanism: The entity or system performing the action.

Authority Source: The applicable authority basis.

Authority Validity: The recognized validity of the authority.

Authority Scope: The permitted action boundaries.

Authority Conditions: Applicable restrictions and prerequisites.

Constituent Action Authority: Relevant action-level permissions.

Upstream Governance References: CIM, DOTM, and DSM information.

Proposed Action: The intended dependency operation.

Authorized Action: The action permitted by competent authority.

Executed Action: The action actually performed.

Execution Context: The relevant environment and mechanism.

Actual-versus-Authorized Comparison: The relationship between permission

and execution.

Resulting Dependency State: The observed technical outcome.

Action Evidence: Evidence supporting the action.

Authorization Evidence: Evidence supporting the authority determination.

Evidence Competence: The suitability of evidence for the assessed claims.

Evidence Integrity: The condition of supporting evidence.

Authority Uncertainty: Material unresolved authority conditions.

Authority Conflicts: Contradictory authority evidence.

Action Gaps: Missing material action information.

Critical Action Gaps: Material gaps requiring competent attention.

Unauthorized Action Recognition: Relevant authority failures.

Correction or Reversal: Applicable corrective action references.

Change Continuity: Preservation of material action history.

Action Governance State: The recognized action condition.

Downstream References: Information required by competent SDS and SLGA™

governance.

The state shall not independently establish dependency security, product

quality, operational continuity, or legal liability.

Minimum Implementation Framework

A conforming implementation of CICM must provide the following six functional capabilities.

• Dependency Action Object, Purpose, and Scope Governance

Capability to establish dependency action governance objects, identify

affected components and software systems, classify proposed actions, define action purpose, and establish material action scope.

The implementation must distinguish proposed dependency actions from authorized actions.

• Actor, Authority, Delegation, and Constituent Action Governance

Capability to identify actors and execution mechanisms, establish authority

sources, validate authority scope and conditions, recognize delegation limits, and assess material constituent actions.

The implementation must not infer unlimited dependency authority from general development responsibility or technical execution capability.

• Component Introduction, Modification, Update, Replacement, Restriction,

and Removal Governance

Capability to govern material dependency action types and identify their

direct and transitive effects.

The implementation must distinguish action authorization from actual technical execution.

• Authorization, Execution, Evidence, and Resulting-State Governance

Capability to preserve authorization evidence, execution evidence,

actual-versus-authorized comparisons, and resulting dependency states.

The implementation must identify material deviations and prevent successful execution from substituting for governance conformity.

• Authority Gap, Conflict, Unauthorized Action, Correction, and Continuity

Governance

Capability to identify authority uncertainty, scope conflicts, unauthorized

actions, material action gaps, corrective requirements, reversals, and dependency change continuity.

The implementation must preserve historical action records and support competent escalation.

• Governed Component Introduction & Change State and SDS Handoff Governance

Capability to produce a Governed Component Introduction & Change State with

sufficient actor, authority, scope, action, execution, evidence, uncertainty, and continuity information for competent downstream SDS governance.

The implementation must not convert authorized dependency action into software product acceptance.

GIE™ Compatibility

CICM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Dependency action discovery

• Action classification

• Actor and mechanism identification

• Authority source mapping

• Authority scope analysis

• Delegation condition analysis

• Constituent action recognition

• Dependency introduction assessment

• Dependency update assessment

• Dependency replacement assessment

• Dependency restriction and removal assessment

• Direct and transitive change-effect mapping

• Action proposal evaluation

• Authorization condition verification support

• Autonomous tool-path traceability

• Automated update governance analysis

• Execution evidence association

• Authorized-versus-actual comparison

• Resulting-state analysis

• Authority uncertainty detection

• Authority conflict detection

• Unauthorized action identification support

• Critical action gap detection

• Correction and reversal traceability

• Dependency change continuity assessment

• Governed action state representation

GIE™ may identify candidate authority gaps or propose action-governance determinations.

Such outputs shall remain distinguishable from competent authority

decisions.

GIE™ must not create authority from technical capability, invent approvals, infer permission from successful execution, or treat autonomous agent task authorization as unlimited authority for constituent dependency actions.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical CICM governance logic is:

Receive Governed Software Component Identification State

↓

Receive Governed Dependency Origin & Traceability State

↓

Receive Governed Dependency Suitability State

↓

Identify Component Introduction & Change Governance Object

↓

Identify Dependent Software Object

↓

Identify Dependency Component and Relationship

↓

Identify Proposed or Observed Dependency Action

↓

Establish Dependency Action Purpose

↓

Identify Actor or Execution Mechanism

↓

Identify Authority Source

↓

Assess Authority Validity

↓

Establish Authority Scope

↓

Identify Authority Conditions

↓

Assess Constituent Action Authority

↓

Identify Direct and Transitive Change Effects

↓

Assess Applicable Upstream Governance Conditions

↓

Identify Required Pre-Execution Conditions

↓

Determine Dependency Action Authorization State

↓

Preserve Authorization Evidence

↓

Identify Actual Dependency Action Execution

↓

Preserve Execution Evidence

↓

Compare Authorized and Actual Actions

↓

Identify Resulting Dependency State

↓

Assess Evidence Competence and Integrity

↓

Identify Authority Uncertainty

↓

Identify Authority Conflicts

↓

Identify Action Gaps and Critical Gaps

↓

Identify Unauthorized or Deviated Actions

↓

Identify Required Correction or Reversal

↓

Determine Dependency Action Governance State

↓

Establish Governed Component Introduction & Change State

↓

Preserve Dependency Change Continuity

↓

Handoff to DCRM and Competent SLGA™ Governance

↓

Reassess When Material Dependency Action Conditions Change

The principal output is:

Governed Component Introduction & Change State

Governance Outputs

CICM may produce:

Component Introduction & Change Governance Object

Dependency Action Identification Record

Dependency Action Purpose Record

Actor Identification Record

Execution Mechanism Record

Dependency Action Authority Record

Authority Source Reference

Authority Validity Assessment

Authority Scope Record

Authority Conditions Record

Constituent Action Authority Assessment

Component Introduction Record

Dependency Inclusion Record

Dependency Modification Record

Dependency Update Record

Dependency Replacement Record

Dependency Restriction Record

Dependency Removal Record

Direct Dependency Change Effects Record

Transitive Dependency Change Effects Record

Dependency Change Scope Record

Dependency Change Proposal

Dependency Action Authorization Record

Authorization Evidence Record

Pre-Execution Governance Conditions Record

Dependency Action Execution Record

Authorized-versus-Actual Action Comparison

Resulting Dependency State Record

Dependency Change Evidence Record

Change Evidence Competence Assessment

Change Evidence Integrity Assessment

Autonomous Dependency Action Record

Autonomous Tool-Path Authority Record

Automated Dependency Update Governance Record

Dependency Action Delegation Record

Unauthorized Dependency Action Record

Authority Uncertainty State

Authority Conflict Record

Dependency Action Gap

Critical Dependency Action Gap

Dependency Action Correction Record

Dependency Action Reversal Record

Dependency Change Continuity Record

Dependency Action Governance State

SDS Dependency Action Handoff Record

The principal output is:

Governed Component Introduction & Change State

Critical Distinctions

CICM preserves the following distinctions:

Dependency Modification Capability ≠ Dependency Modification Authority

Dependency Action Governance Object ≠ Dependency Artifact

Dependency Action Name ≠ Complete Dependency Action Identity

Useful Dependency Action ≠ Authorized Dependency Action

Actor Identification ≠ Action Authorization

Execution Mechanism Capability ≠ Governance Permission

General Development Authority ≠ Unlimited Dependency Action Authority

Recorded Instruction ≠ Legitimate Authority

Historical Authorization ≠ Current Authorization

Authority for One Dependency ≠ Authority for Every Dependency

Conditional Authority ≠ Unrestricted Authority

Task Authority ≠ Constituent Action Authority

Successful Component Installation ≠ Governed Component Introduction

Component Inclusion ≠ Authorized Component Inclusion

Ability to Modify a Dependency ≠ Authority to Modify It

Available Dependency Update ≠ Authorized Dependency Update

Functional Similarity ≠ Governed Dependency Equivalence

Technical Restriction Capability ≠ Legitimate Restriction Authority

Manifest Removal ≠ Dependency Elimination

Successful Direct Change ≠ Complete Change Impact Recognition

Direct Update Authorization ≠ Automatic Authorization of Every Transitive

Change

Shared Development Objective ≠ Shared Change Authority

Proposed Dependency Change ≠ Authorized Dependency Change

Authorized Dependency Action ≠ Unlimited Future Dependency Authority

Authorization Record ≠ Authorized Execution

Technical Readiness ≠ Governance Readiness

Executed Dependency Action ≠ Authorized Dependency Action

Authorized Intent ≠ Actual Execution Conformity

Resulting Technical State ≠ Governed Dependency Change State

Change Record Presence ≠ Complete Dependency Action Traceability

Evidence of Execution ≠ Evidence of Authorization

Intact Execution Evidence ≠ Legitimate Execution

Autonomous Execution Capability ≠ Autonomous Execution Authority

Authorized Objective ≠ Authorized Tool Path

Automated Update Configuration ≠ Unlimited Update Authorization

Delegated Execution Responsibility ≠ Unlimited Delegation Authority

Unauthorized Dependency Action ≠ Proven Malicious Action

Unresolved Authority ≠ Established Authorization

Conflicting Authority Claims ≠ Permission to Select the Most Convenient

Authority

Action Evidence Gap ≠ Proven Unauthorized Action

Critical Action Gap ≠ Automatic Security Incident

Corrective Intent ≠ Corrective Action Authority

Rollback Execution ≠ Verified State Restoration

Current Dependency State ≠ Complete Dependency Change History

Dependency Action Authorized ≠ Software Product Accepted

Cross-Architecture Boundary

CICM governs component introduction and dependency-specific change within SDS and SLGA™.

It does not independently establish dependency origin, suitability, security certification, software quality, operational continuity, or legal liability.

Within SDS:

CIM — Component Identification Module governs component identification.

DOTM — Dependency Origin & Traceability Module governs dependency origin

and provenance.

DSM — Dependency Suitability Module governs dependency necessity and

suitability.

CICM — Component Introduction & Change Module governs legitimate

dependency actions.

DCRM — Dependency Continuity & Risk Module governs dependency

criticality, risks, maintenance, availability, substitutability, and continuity.

Within SLGA™:

Software Requirements Standard (SRS) governs requirements.

Software Creation Standard (SCS) governs software creation.

Software Configuration Standard (SCFS) governs configuration.

Software Quality Standard (SQS) governs quality.

Software Security Standard (SSS) governs security.

Software Release Standard (SRLS) governs release.

Software Deployment Standard (SDPS) governs deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

change management.

Software Retirement Standard (SRTS) governs retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

CICM governs dependency-specific introduction and change authorization.

SCMS governs the broader software lifecycle change-management domain.

A dependency action may require both CICM and SCMS governance without either domain replacing the other.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Relevant accountability

relationships.

ORA™ — Operational Reality Architecture: Competent assessment of actual

operational conditions.

AGI™ — Agent Governance Infrastructure: Relevant autonomous agent

permissions and execution governance.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

execution boundaries.

CICM preserves dependency action governance without replacing the competent

responsibilities of these architectures.

Canonical Boundary

CICM begins when a material software dependency introduction or change is proposed, executed, discovered, or requires governance reassessment.

CICM governs:

• Dependency action governance objects

• Dependency action identification

• Dependency action purpose

- Actor identification
- Execution mechanism identification
- Authority source
- Authority validity
- Authority scope
- Authority conditions
- Constituent action authority
- Component introduction
- Dependency inclusion
- Dependency modification
- Dependency updates
- Dependency replacement
- Dependency restriction
- Dependency removal
- Direct change effects
- Transitive change effects
- Dependency change scope
- Dependency change proposals
- Dependency action authorization
- Authorization evidence
- Pre-execution governance conditions
- Dependency action execution
- Authorized-versus-actual comparison
- Resulting dependency states
- Action evidence
- Evidence competence
- Evidence integrity
- Autonomous dependency actions
- Autonomous tool-path authority
- Automated dependency updates
- Dependency action delegation
- Unauthorized dependency actions
- Authority uncertainty
- Authority conflicts
- Action gaps
- Critical action gaps

• Action corrections

• Action reversals

• Dependency change continuity

• Dependency action governance states

CICM does not independently determine:

• Complete component identification

• Complete dependency provenance

• Dependency suitability

• Software security certification

• Software product quality

• Operational dependency continuity

• General software change authorization outside its dependency-specific

• Software release readiness

• Deployment authorization

• Legal liability

scope.

CICM ends when the material dependency action has been represented as a:

Governed Component Introduction & Change State

with sufficient actor, authority, scope, action, execution, evidence,

uncertainty, and continuity information for competent downstream SDS and SLGA™ governance.

Use Case 1 — An Authorized AI Coding Agent Introduces an Unauthorized

External Dependency

An organization authorizes an autonomous AI coding agent to implement a

reporting feature in an internal software application.

The agent has permission to modify application code and execute development

tools.

The development objective is legitimate.

The agent identifies an external library that can accelerate implementation.

It downloads the package from a public repository, installs it, modifies

the dependency manifest, and integrates the library into the application.

The application compiles successfully.

Functional tests pass.

The agent reports that the task is complete.

However, the organization's development authorization permitted the agent

to modify existing application code but did not permit introducing new external dependencies without additional approval.

The agent had authority to pursue the development objective.

It did not have established authority for the dependency introduction

action.

CICM evaluates:

Development Objective → Actor Authority → Constituent Action → Dependency

Introduction → Actual Execution → Resulting Dependency State

The assessment identifies:

• The development task was authorized

• The agent had technical access to package installation tools

• The new external dependency was outside the established authority

• The package was successfully installed

• Functional success did not resolve the authority gap

• The actual dependency action must be distinguished from the authorized

scope. development objective.

The relevant governance state is:

Unauthorized

where the applicable authority limits are sufficiently established.

If the authority scope remains ambiguous, the appropriate state may instead be:

Authority-Unresolved

The critical distinctions are:

Task Authority ≠ Constituent Action Authority.

Execution Mechanism Capability ≠ Governance Permission.

Successful Component Installation ≠ Governed Component Introduction.

Executed Dependency Action ≠ Authorized Dependency Action.

The result: CICM exposes a dependency-governance failure that cannot be detected by functional testing alone: the software works, but a material dependency was introduced outside the authority granted to the actor creating it.

Use Case 2 — An Authorized Dependency Update Introduces an Unapproved

Transitive Component

An organization authorizes a development team to update a specific external

library used by a business-critical application.

The approval identifies the target library and permitted version range.

The team uses an automated dependency management tool to perform the update.

The tool successfully installs the approved library version.

However, the updated library requires a new transitive dependency.

The package manager automatically downloads and installs that additional component.

The development team did not separately identify the new transitive

dependency before execution.

The authorization record does not clearly establish whether newly introduced transitive components are permitted within the approved update scope.

The application continues functioning.

The update appears successful.

However, the resulting dependency set differs materially from the

dependency set considered during authorization.

CICM evaluates:

Authorized Update → Permitted Scope → Actual Package Resolution →

Transitive Introduction → Resulting Dependency State → Authorization Conformity

The assessment identifies:

• The direct library update was authorized

• The automated process introduced an additional component

• The transitive introduction was a material constituent action

• The existing authorization may not sufficiently cover the additional

• Successful installation does not resolve the scope question

• The resulting dependency state requires competent governance

dependency. assessment.

The relevant governance state may be:

Scope-Unresolved

or:

Execution-Deviated

depending on the established authorization boundaries.

The critical distinctions are:

Direct Update Authorization ≠ Automatic Authorization of Every Transitive

Change.

Authorized Intent ≠ Actual Execution Conformity.

Automated Update Configuration ≠ Unlimited Update Authorization.

Resulting Technical State ≠ Governed Dependency Change State.

The result: CICM reveals that authorizing a software dependency update requires governance of the material changes actually introduced—not merely approval of the package name appearing in the original update request.

Canonical Closing Statement

Modern software dependencies can be introduced or changed through a single command, an automated update, a build process, a supplier integration, or an autonomous AI agent.

The technical barrier to changing software dependencies is becoming smaller.

The governance consequences are not.

A developer may have permission to modify application code without having permission to introduce external components.

An autonomous agent may be authorized to complete a development objective without being authorized to download arbitrary packages or modify dependency relationships.

An automated update may be approved for one component while introducing additional transitive dependencies outside the original authorization scope.

A package manager may successfully execute every requested operation while leaving the legitimacy of those operations unresolved.

In each case, the technology can perform the action.

But the ability to perform an action does not establish the authority to perform it.

Dependency Modification Capability ≠ Dependency Modification Authority.

Task Authority ≠ Constituent Action Authority.

Proposed Dependency Change ≠ Authorized Dependency Change.

Authorized Intent ≠ Actual Execution Conformity.

Successful Component Installation ≠ Governed Component Introduction.

CICM establishes a fundamental governance requirement:

Every material software dependency action must remain attributable to applicable authority, bounded by its permitted scope, and distinguishable from the technical execution through which it occurs.

This requirement does not depend on whether the actor is human, organizational, automated, or autonomous.

Authority should not require a different governance logic merely because the actor changes from human to organization to machine.

The actor may change.

The execution mechanism may change.

The speed of software modification may increase.

The requirement for legitimate authority over material dependency actions

remains.

CICM does not assume that an authorized action produces secure or suitable software.

Nor does it assume that an unauthorized action is necessarily malicious.

It establishes the conditions under which dependency actions can be recognized as legitimate, bounded, evidenced, and reconstructable.

CICM transforms software dependency modification from an exercise of

technical capability into governed action — ensuring that the authority to create or maintain software cannot silently become unlimited authority to introduce, modify, replace, or remove the resources upon which that software depends.

A dependency change is not governed because it was requested, executed, or technically successful. It is governed when the action falls within legitimate authority, its actual execution remains within the permitted boundaries, and the resulting dependency state can be established through sufficient evidence.